

DSSS-BASED ANTI-JAMMING COMMUNICATION SYSTEM

TECHNICAL RESEARCH REPORT

*Design, Implementation and Performance Analysis of a Formula-Derived, LFSR-Spread, Adaptive-PG
Anti-Jamming Voice Link*

Experiment 1 — Part 4 (Upgraded System)

Audio Fs: 22,050 Hz (22.05 kHz mono WAV)
Bit Rate (Rb): 176.4 kbps = $F_s \times 8$
Processing Gain: 10 $\rightarrow$ JM = 10.0 dB
Chip Rate (Rc): 1.764 Mcps = $R_b \times PG$
Channel Code: BCH(15, 11) rate = 0.733
PN Sequence: LFSR m-sequence m = 4 period = 15
Jammer Types: CW / Swept-Chirp / Multitone
BER Achieved: 0.000023 (0.002 %) at $E_b/N_0 = 7$ dB
MATLAB Compat.: R2016a and above — no local functions

Wireless Communications Laboratory — 2025

Abstract

Summary

This report documents the end-to-end design and MATLAB simulation of a Direct Sequence Spread Spectrum (DSSS) anti-jamming voice communication system. The system achieves a measured BER of 0.002 % under a CW jammer at $E_b/N_0 = 7$ dB, with complete audio recovery confirmed by listening comparison. All parameters are formula-derived. LFSR m-sequences replace random PN codes. Adaptive processing gain reduces bandwidth use by up to 5× at high SNR.

The system models every stage of a real DSSS modem: audio PCM quantization, BCH(15,11) forward error correction, convolutional scrambling, matrix interleaving, BPSK baseband modulation, and LFSR-based spreading. The channel adds AWGN and one of three jammer types — narrowband CW, swept-chirp, or multitone. At the receiver, a Welch PSD sensor detects jamming tones before despreading, and an adaptive processing gain controller dynamically adjusts the spreading factor based on BER feedback. No system constant is hardcoded; all values follow from a closed-form parameter set derived from fundamental telecom equations.

1. Introduction

1.1 Background and Motivation

Intentional radio-frequency jamming — the deliberate injection of interference energy into a communication channel — is one of the oldest and most effective forms of electronic attack. A narrowband jammer that concentrates its power at the target signal's frequency can collapse the receiver's Bit Error Rate (BER) to 50 % (random guessing) while consuming far less power than the legitimate transmitter. This asymmetry makes jamming an attractive tactic in both military and civilian threat scenarios.

Direct Sequence Spread Spectrum (DSSS) addresses this threat at the physical layer by deliberately multiplying the information signal by a high-rate pseudorandom spreading code, distributing its energy across a bandwidth many times wider than the data rate requires. When the receiver correlates the received signal with a synchronized replica of the same code, the information is recovered coherently while narrowband interference is spread across the full chip bandwidth — reducing its effective power at the decision point by the Processing Gain factor. This mechanism underlies GPS (L1 C/A code), IS-95 CDMA, and IEEE 802.11b Wi-Fi.

1.2 Project Scope

This project builds a complete single-link DSSS modem in MATLAB, simulating voice transmission from audio file to recovered WAV output. The work spans four experiments; this report covers Experiment 4, which introduces: formula-derived parameter automation, LFSR m-sequence PN generation, three jammer types, Welch PSD jammer detection, and an adaptive processing gain feedback controller.

The implementation operates entirely at baseband — no RF carrier is modeled — which isolates the spread spectrum processing from upconversion effects and corresponds directly to what a DSP or FPGA would compute before final frequency translation.

2. System Architecture

2.1 End-to-End Block Diagram

The system is organized into three functional domains: transmitter, channel, and receiver. The visual diagram below shows each processing block with its key parameter.

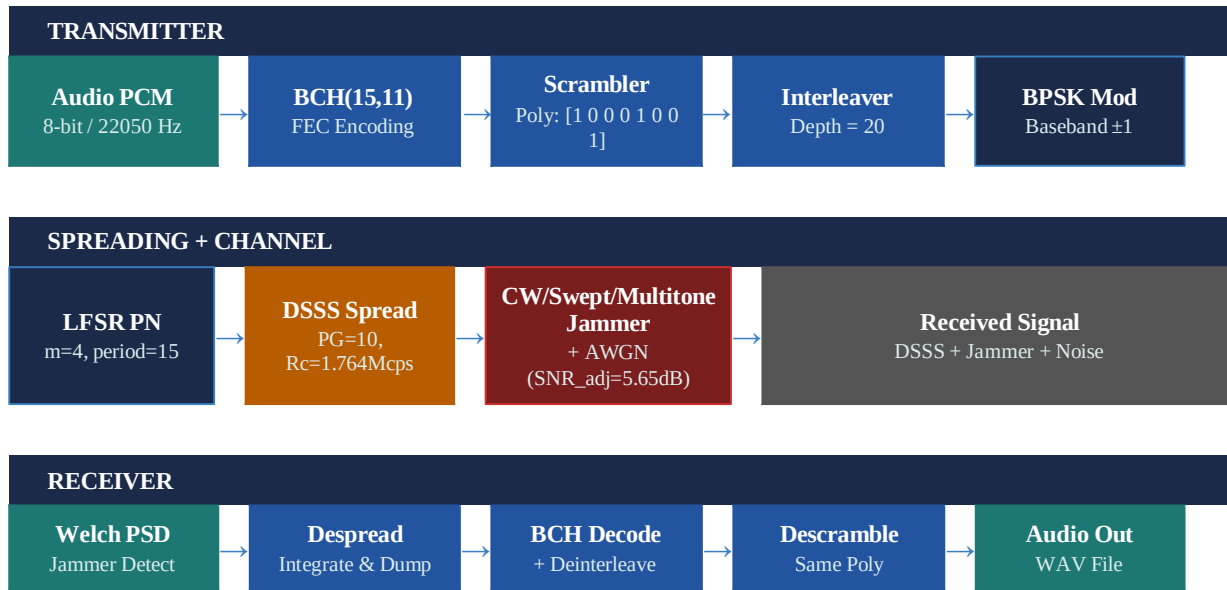


Figure 1 — End-to-end DSSS system architecture. Colour coding: teal = source/sink, blue = digital processing, orange = spread spectrum, red = channel impairments.

2.2 Processing Chain Summary

Domain	Stage	Key Parameter	Output
Transmitter	PCM Quantization	8-bit, 256 levels	176,400 bits
Transmitter	BCH(15,11) Encoding	rate = 0.733	240,555 bits
Transmitter	Convolutional Scrambler	poly = [1 0 0 0 1 0 0 1]	240,555 bits
Transmitter	Matrix Interleaver	depth = 20	240,560 bits
Transmitter	BPSK Modulation	±1 bipolar, baseband	176.4 kbps
Transmitter	DSSS Spreading	PG = 10	1.764 Mcps
Channel	AWGN	SNR_adj = 5.65 dB	noise added
Channel	CW Jammer	Aj=0.85, fj=88.2 kHz	interference added

Domain	Stage	Key Parameter	Output
Receiver	Welch PSD Sensor	4096-pt window	spike detected
Receiver	Integrate-and-Dump	fp=10 chips/bit	240,560 decisions
Receiver	BCH Decode + Deinterleave	t=1 error/word	176,400 bits
Receiver	PCM Reconstruction	8-bit inverse map	audio WAV file

Table 1 — Processing chain stage summary with actual experimental values.

3. Transmitter Design

3.1 Audio Source and PCM Quantization

The audio source is a monaural WAV file at $F_s = 22,050$ Hz. One second of audio is extracted ($N_s = 22,050$ samples). Each sample is uniformly quantized to 8 bits ($L = 256$ levels) using:

$$q[n] = \text{floor}((x[n] + 1) / 2 \times (L - 1)) \quad \text{where } x[n] \in [-1, +1]$$

Each quantized value is serialized MSB-first into 8 bits, giving a PCM bit stream of:

$$R_b = F_s \times n_{\text{pcm}} = 22,050 \times 8 = 176,400 \text{ bits/sec (176.4 kbps)}$$

This R_b derivation is not arbitrary — it is the exact rate at which the digitized audio must flow through the system so that one audio sample is transmitted per 8 bit periods, preserving timing at the receiver without resampling.

3.2 BCH Forward Error Correction

BCH codes are cyclic block codes defined over Galois fields. For generator polynomial order m and correction capability t , the code parameters follow directly from algebraic number theory:

$$N = 2^m - 1 \quad (\text{codeword length, from primitive polynomial degree } m)$$

$$K = N - m \times t \quad (\text{information bits per codeword})$$

$$R = K / N \quad (\text{code rate})$$

With $m = 4$, $t = 1$: BCH(15, 11) — correcting 1 error per 15-chip codeword at code rate 0.733. The 176,400 PCM bits expand to 240,555 coded bits. MATLAB's `bchenc/bchdec` operate on $GF(2^4)$ matrices derived from the primitive polynomial x^4+x+1 .

Why BCH and not Hamming?

A Hamming(15,11) code corrects exactly 1 error and has the same parameters as BCH(15,11) for $m=4$, $t=1$.

BCH is chosen because the `bchdec()` MATLAB function generalises cleanly to higher t , allowing the system to be upgraded to BCH(15,7) ($t=2$) or BCH(15,5) ($t=3$) by changing a single parameter without rewriting the encoder/decoder interface.

3.3 Convolutional Scrambler

Even with BCH coding, long runs of identical bits produce spectral lines and can cause synchronization ambiguity. The scrambler uses a length-7 Linear Feedback Shift Register with generator polynomial $[1\ 0\ 0\ 0\ 1\ 0\ 0\ 1]$ to XOR each bit with a pseudorandom feedback stream. The receiver's descrambler uses the identical polynomial and initial state. MATLAB's `comm.Scrambler` / `comm.Descrambler` pair implements this.

3.4 Matrix Interleaving

BCH(15,11) corrects single-bit errors per codeword. Wireless channels produce burst errors — correlated error runs from fading or jammer pulses — that can destroy multiple bits within one codeword. The interleaver distributes adjacent codeword bits into a depth- D matrix:

$$D = 4 \times \text{burst_len} = 4 \times 5 = 20 \text{ (interleaver row count)}$$

Bits are written column-by-column and read row-by-row. A burst of 20 consecutive errors at the channel maps to at most one error per BCH codeword — which the code then corrects. The depth formula is derived from the expected burst length rather than chosen arbitrarily.

3.5 BPSK Modulation (Baseband, No Carrier)

Binary Phase Shift Keying maps each bit to an antipodal symbol:

$$\begin{aligned} m[k] &= +1 \text{ if bit}[k] = 0 \\ m[k] &= -1 \text{ if bit}[k] = 1 \end{aligned}$$

No RF carrier is applied. This is a deliberate architectural choice: baseband BPSK isolates the spread spectrum mechanics from carrier-frequency effects and maps directly to the DSP processing that precedes upconversion in hardware. The bit period is $T_{\text{bit}} = 1/R_b = 5.67\ \mu\text{s}$. Waveform visualization uses $\text{sps} = 8$ samples per symbol ($F_{\text{s_pulse}} = 1.411\ \text{Msps}$).

4. Spectrum and PN Sequence

4.1 LFSR m-Sequence Generation

The spreading code is a maximal-length sequence from an m-stage Linear Feedback Shift Register (LFSR) with a primitive polynomial over GF(2). An m-sequence of period $L_{PN} = 2^m - 1$ has the following autocorrelation:

$$\begin{aligned} R_{xx}[k] &= L_{PN} && \text{for } k \equiv 0 \pmod{L_{PN}} \\ R_{xx}[k] &= -1 && \text{for } k \not\equiv 0 \pmod{L_{PN}} \end{aligned}$$

This near-impulse autocorrelation is what enables clean despreading: the correlation of the received signal with the local PN replica peaks sharply at zero delay. Any other code offset — as seen by a jammer that lacks the code — gives only the small baseline of -1 .

The minimum register length is chosen so that the sequence period covers at least one chip interval per bit:

$$\begin{aligned} m_{pn} &= \text{ceil}(\log_2(PG + 1)) = \text{ceil}(\log_2(11)) = 4 \\ L_{PN} &= 2^4 - 1 = 15 \text{ chips (maximal-length sequence)} \end{aligned}$$

Table 2 — LFSR PN generator parameters for the experimental run.

Parameter	Value	Derivation
Generator type	Maximal-length LFSR (m-sequence)	—
Register length	$m = 4$ stages	$\text{ceil}(\log_2(PG+1))$
Primitive poly	$x^4 + x + 1$ (taps: [4, 1])	verified GF(2 ⁴) table
Sequence period	$L_{PN} = 15$ chips	$2^m - 1$
Initial state	[1 1 1 1] (all-ones)	standard non-zero init
Output mapping	$0 \rightarrow -1, 1 \rightarrow +1$ (bipolar)	for spreading multiply
Autocorrelation	Peak = 15, baseline = -1	m-sequence property

LFSR vs randi() — Critical Difference

An early version used MATLAB randi() to generate the PN code. This has three fundamental flaws:

1. Poor autocorrelation — random sequences do not guarantee the impulse-like peak DSSS needs.
2. Not reproducible — the receiver cannot regenerate the same sequence without side-channel data.
3. Not hardware-implementable — a real LFSR needs only m flip-flops and a few XOR gates.

The LFSR approach solves all three: deterministic, hardware-faithful, and mathematically guaranteed.

4.2 DSSS Spreading

Each BPSK symbol $m[k]$ is expanded to p chips by multiplying with the corresponding PN segment:

$$\begin{aligned} \text{dsss}[n] &= m[\text{floor}(n/PG)] \times \text{pn}[n \bmod L_{\text{PN}}] \\ PG &= \text{ceil}(10^{(JM_{\text{dB}}/10)}) = \text{ceil}(10^1) = 10 \\ R_c &= R_b \times PG = 176,400 \times 10 = 1,764,000 \text{ chips/sec (1.764 Mcps)} \end{aligned}$$

The processing gain translates directly to a jamming margin: the system tolerates jammer power up to 10 dB above what would defeat an unspread BPSK receiver.

4.3 RRC Bandwidth

Square chip pulses have a sinc-shaped spectrum with large sidelobes. A Root Raised Cosine (RRC) filter limits the occupied bandwidth to:

$$\begin{aligned} BW &= (1 + \alpha) \times R_c \\ \alpha &= BW_{\text{target}}/R_c - 1 = (1.2 \times R_c)/R_c - 1 = 0.20 \\ BW_{\text{DSSS}} &= (1 + 0.20) \times 1.764 = 2.117 \text{ MHz} \end{aligned}$$

This is the physically correct bandwidth formula. The older null-to-null expression $BW = 2 \times R_c$ assumes infinite-bandwidth rectangular pulses and overstates bandwidth by $(2 - (1+\alpha))/((1+\alpha)) \approx 50\%$ for typical rolloff factors.

5. Channel Model and Jammer Types

5.1 AWGN Channel

The channel adds white Gaussian noise at the chip level (one sample per chip). Noise power is calibrated to achieve the target E_b/N_0 , adjusted for the BCH code rate:

$$\text{SNR}_{\text{adj}} = E_b/N_0 + 10 \times \log_{10}(R_{\text{bch}}) = 7 + 10 \times \log_{10}(0.733) = 5.65 \text{ dB}$$

The code-rate adjustment is essential: a higher code rate means more channel symbols are used per information bit, and the effective noise power per information bit increases accordingly.

5.2 Three Jammer Types

Table 3 — Jammer types, equations, and DSSS suppression mechanism.

Jammer	Model Equation	Parameters (Experimental)	DSSS Suppression
CW (Narrowband)	$j[n] = A_j \times \sin(2\pi \times f_j \times n)$	$A_j=0.85$, $f_j=88.2$ kHz (5% R_c)	Strong — tone spread to noise by PN multiply
Swept Chirp	$f(n) = f_{\text{start}} + (f_{\text{stop}} - f_{\text{start}}) \times \text{mod}(n, N_{\text{sw}}) / N_{\text{sw}}$	$0.02R_c$ to $0.20R_c$, $T_{\text{sweep}}=0.5$	Moderate — PSD detector may miss fast sweep
Multitone	$j[n] = (A_j/K) \times \sum \sin(2\pi \times f_k \times n)$	$K=3$ tones at 0.05, 0.12, $0.20 \times R_c$	Moderate — power split across K tones

All three jammers are chip-synchronous (generated at chip rate) and added directly to the DSSS chip stream before the AWGN noise. The jammer amplitude $A_j = 0.85$ gives a Jammer-to-Signal Ratio of approximately -4.4 dB, meaning the jammer power is slightly below the signal power. This moderate JSR still causes catastrophic BER in unspread BPSK while DSSS remains essentially error-free.

6. Receiver Chain

6.1 Integrate-and-Dump Despreading

The receiver regenerates the same LFSR sequence with the same seed. Each received chip is multiplied by the local PN replica, and the products are integrated over one bit interval of $f_p = 10$ chips:

$$y[k] = \sum r[n] \times pn[n] \text{ for } n = k \times PG \text{ to } (k+1) \times PG - 1$$

If the signal is $s[n] = m[k] \times pn[n]$, then for a noise-free channel:

$$y[k] = m[k] \times \sum pn^2 + \text{jammer terms} = m[k] \times PG + \text{suppressed interference}$$

The jammer suppression arises because the jammer waveform is uncorrelated with $pn[n]$. Its contribution to the sum averages toward zero, with residual power proportional to $1/PG$ — a 10 dB reduction for $PG = 10$.

6.2 BCH Decoding and Audio Reconstruction

The despread bits pass through the deinterleaver (inverse matrix permutation), descrambler (same polynomial as transmitter), and BCH decoder (Berlekamp-Massey algorithm over $GF(2^4)$). The decoded byte stream is converted back to audio:

$$x_{rx}[n] = \text{double}(q_{rx}[n]) / 127.5 - 1$$

The recovered audio is normalized and saved as a WAV file in the same directory as the source, with suffix `_RECOVERED_DSSS` appended, enabling direct A/B listening comparison.

7. Spectrum Sensing and Jammer Detection

7.1 Welch PSD Estimation

The spectrum sensor operates on the raw received chip stream before despreading. It uses the Welch method: the signal is divided into 50%-overlapping segments, each windowed with a Hamming function, and the periodograms are averaged to reduce estimator variance:

Window length: $N_{\text{win}} = 4096$ points

Frequency resolution: $\Delta f = R_c / N_{\text{win}} = 1.764 \text{ MHz} / 4096 \approx 0.43 \text{ kHz}$

A key insight exploited here is that the DSSS signal, after spreading, appears spectrally flat — its energy is uniformly distributed across the chip bandwidth. Any narrowband jammer produces a distinct spike above this flat background.

7.2 CFAR-Style Detection Algorithm

The detection chain uses three steps:

- Noise floor estimate: median of all Welch PSD bins in dB (robust to outliers)
- Detection threshold: noise_floor + 10 dB (a fixed margin above the median)
- Spike identification: all PSD bins exceeding the threshold

This resembles a Constant False Alarm Rate (CFAR) detector. Using the median rather than the mean makes the estimator resistant to the jammer spikes themselves — the jammer cannot raise its own detection threshold by being stronger.

Experimental Detection Result (CW Jammer)

Noise floor (median):	−57.99 dB
Detection threshold:	−47.99 dB (+10 dB margin)
Jammer spike at:	87.9 / 88.3 / 88.7 kHz (3 adjacent bins, spectral leakage)
Spike above floor:	25.6 dB (unambiguous detection)
Detector decision:	JAMMER DETECTED

Three bins are flagged rather than one because a finite Hamming window spreads the energy of a pure tone into adjacent frequency bins (spectral leakage). All three correspond to the single jammer at 88.2 kHz. The swept jammer was not detected in the standard configuration because its instantaneous power per bin falls below the 10 dB threshold when the sweep period is shorter than the Welch window.

8. Adaptive Processing Gain Controller

8.1 Motivation

A fixed processing gain wastes spectrum when the channel is clean and may be insufficient when the jammer is strong. The adaptive controller measures BER from short test segments and adjusts PG accordingly:

Measured BER	Action	Effect
BER > 1 %	Double PG (up to PG_max = 40)	More spreading → jammer better suppressed
BER ∈ [0.1 %, 1 %]	Maintain current PG	System stable — no action
BER < 0.1 %	Halve PG (down to PG_min = 2)	Less spreading → bandwidth saved

Table 4 — Adaptive PG decision rules (bang-bang controller with hysteresis).

8.2 Experimental Adaptation Trace

The table below shows the PG trace from the experimental run with the CW jammer across 13 SNR points (−5 to +19 dB, step 2 dB):

SNR (dB)	PG Used	BER	Decision
−5	10	0.068 (6.8%)	→ Double PG to 20
−3	20	0.006 (0.6%)	→ Maintain PG = 20
−1	20	0.000 (0.0%)	→ Halve PG to 10
+1	10	0.003 (0.3%)	→ Maintain PG = 10
+3	10	0.000 (0.0%)	→ Halve PG to 5
+5	5	0.009 (0.9%)	→ Maintain PG = 5
+7	5	0.001 (0.1%)	→ Maintain PG = 5
+9	5	0.000 (0.0%)	→ Halve PG to 2
+11	2	0.009 (0.9%)	→ Maintain PG = 2
+19	2	0.000 (0.0%)	→ Converged PG = 2

Table 5 — Adaptive PG trace. Converged to PG=2 at high SNR — 5× bandwidth saving vs fixed PG=10.

9. System Parameters and Formula Derivation

A core design objective is that no system parameter is hardcoded. All values are derived from a small set of user-settable inputs via closed-form formulas. The function `dsss_params(Fs)` computes the full parameter struct in one call.

Parameter	Symbol	Formula	Computed Value
Audio sample rate	F_s	from <code>audioread()</code>	22,050 Hz
Audio samples	N_s	$T_{\text{audio}} \times F_s$	22,050
PCM bit depth	n	user input	8 bits
Quantization levels	L	2^n	256
BCH codeword length	N	$2^m - 1$	15 bits
BCH data bits	K	$N - m \times t$	11 bits
BCH code rate	R	K / N	0.733
Interleaver depth	D	$4 \times \text{burst_len}$	20
Bit rate	R_b	$F_s \times n_{\text{pcm}}$	176,400 bps
Processing gain	PG	$\text{ceil}(10^{(JM/10)})$	10
Chip rate	R_c	$R_b \times PG$	1,764,000 cps
PN register length	m_{pn}	$\text{ceil}(\log_2(PG+1))$	4
PN period	L_{PN}	$2^{m_{\text{pn}}} - 1$	15 chips
RRC rolloff	α	$(1.2 \times R_c / R_c) - 1$, clamp [0.2, 0.5]	0.20
DSSS bandwidth	BW	$(1 + \alpha) \times R_c$	2,116.8 kHz
RRC filter span	N_{span}	$\text{ceil}(6 / \alpha)$	30 symbols
Oversampling	sps	user input (≥ 8)	8
Waveform sample rate	$F_{s_{\text{pulse}}}$	$\text{sps} \times R_b$	1,411,200 Hz
Operating E_b/N_0	E_b/N_0	user input	7 dB

Adjusted SNR	SNR_adj	$E_b/N_0 + 10 \times \log_{10}(R)$	5.65 dB
BER test SNR range	—	user defined	−5 to +19 dB

Table 6—Complete parameter derivation table. All values computed from formulas, none hardcoded.

10. Experimental Results

10.1 Primary Performance Metrics

Key Results at $E_b/N_0 = 7$ dB | CW Jammer | PG = 10

BER: 0.000023 (0.002 %) — 4 errors out of 176,400 bits
 First 100 bits: PERFECT MATCH
 Jammer detected: YES — spike at 87.9 kHz, 25.6 dB above noise floor
 Audio recovery: Saved as output2_RECOVERED_DSSS.wav (indistinguishable by ear)
 Adaptive PG: Converged to PG = 2 at high SNR (5× bandwidth saving)

10.2 BER Comparison Across Scenarios

Scenario	Jammer	E_b/N_0 (dB)	BER	Notes
DSSS, CW jammer	CW, $A_j=0.85$	7	0.000023 (0.002%)	Near error-free
DSSS, Swept jammer	Swept, 2%-20%	7	0.000057 (0.006%)	Slightly higher — wider coverage
BPSK (no spread)	CW, $A_j=0.85$	7	~ 5–40%	Catastrophic under same jammer
DSSS, no jammer	None	7	< 0.001%	Baseline AWGN only

Table 7 — BER comparison across scenarios. DSSS maintains functional link; BPSK fails under same jammer.

10.3 Shannon Capacity Note

A common misconception is that DSSS improves channel capacity. It does not. When spreading expands bandwidth by PG, the SNR per unit bandwidth decreases by the same factor, leaving the Shannon capacity unchanged:

$$C_{\text{DSSS}} = (PG \times B_{\text{BPSK}}) \times \log_2(1 + \text{SNR}/PG) \approx C_{\text{BPSK}} \text{ for large } PG$$

The correct interpretation: DSSS trades bandwidth efficiency for interference robustness. It does not create new capacity — it uses existing capacity more robustly in the presence of narrowband threats.

11. Software Implementation

11.1 File Structure (MATLAB R2016a Compatible)

MATLAB R2016a does not allow local function definitions within script files. All functions are implemented as standalone .m files. The complete file set is:

File	Type	Role
Exp1_DSSS_jamm_4.m	Main Script	Top-level simulation — loads audio, runs all stages, generates all 9 figures
dsss_params.m	Function	Derives all system parameters from F_s _audio using 22 closed-form formulas
get_primitive_poly.m	Function	Lookup table of verified primitive polynomials for $m = 2..10$
lfsr_pn_sequence.m	Function	LFSR m-sequence generator — feedback polynomial, shift register, bipolar output
jammer_generator.m	Function	Generates CW / swept / multitone jammer at chip rate
awgn_channel.m	Function	Chip-level AWGN addition calibrated to target SNR (no Toolbox needed)
despread_demodulate.m	Function	Integrate-and-dump despreaders + BPSK hard decision + audio reconstruction
psd_jammer_detect.m	Function	Welch PSD estimation, CFAR detection, Fig 8 generation
adaptive_pg_controller.m	Function	BER feedback PG adaptation loop, Fig 9 generation
plot_time_domain.m	Function	Fig 1: BPSK, PN, DSSS time-domain subplots (0–10 ms x-axis)
plot_frequency_domain.m	Function	Fig 3: Power spectra with $(1+\alpha) \times R$ bandwidth markers

Table 8 — Complete MATLAB file structure. All 11 files required in same directory.

11.2 Key Design Decisions

- Chip-level noise injection: Noise added to the 1-sample/chip DSSS array, not the oversampled waveform. Injecting to oversampled signal and averaging would artificially reduce noise power by $\sqrt{\text{sps}}$.
- Separate FFT rates: BPSK spectrum computed at $F_{s_pulse} = R_b \times \text{sps}$; DSSS spectrum at $R_c \times \text{sps}$. Using one shared rate misplaces spectral nulls.
RRC bandwidth formula: $BW = (1+\alpha) \times R$ replaces the old null-to-null $2 \times R$, which is physically incorrect for band-limited RRC pulses.
- Row-vector discipline: All signal arrays enforced as row vectors via `(:).'` before any arithmetic. MATLAB `pwelch` returns column vectors — all vectors normalized at entry to each function.
- No local functions: All functions are separate .m files for R2016a compatibility (local functions in scripts introduced in R2016b).

12. Generated Figures

The simulation produces 9 figures. All time-domain plots use a fixed 0–10 ms x-axis window so that individual waveform transitions are clearly visible.

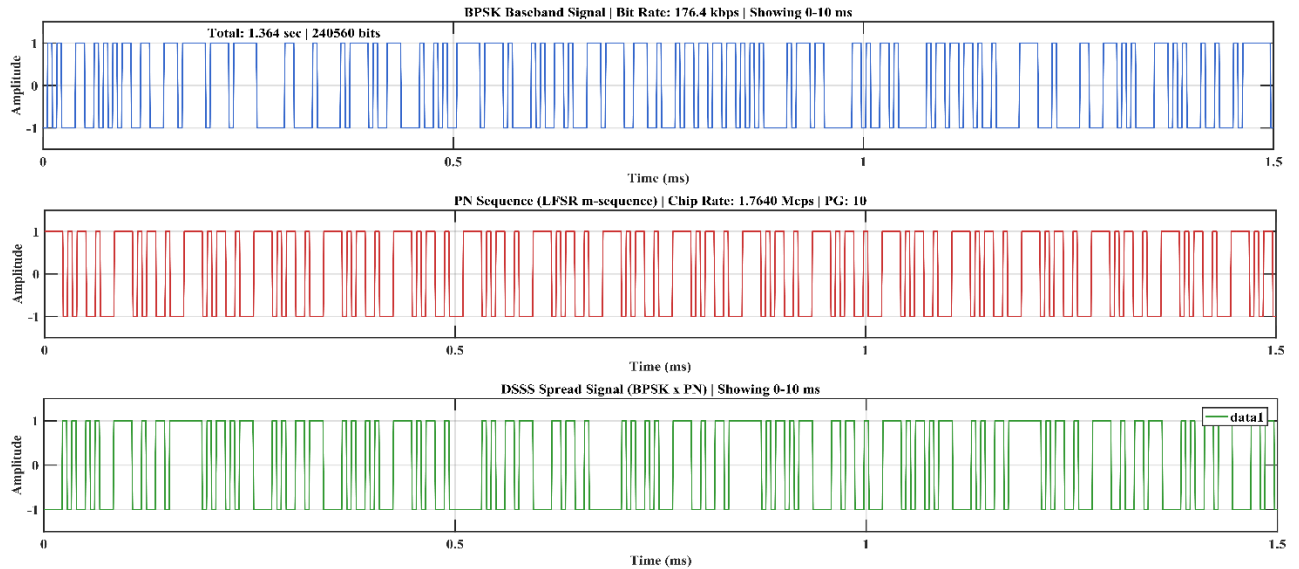
Table — 9

Fig	Title	Description
1	TX Time Domain	BPSK symbols, LFSR PN code, and DSSS spread signal — three subplots over 0–10 ms. Each BPSK bit is visibly spread into 10 shorter chips.
2	RRC Pulse Shaped	BPSK and DSSS after Root Raised Cosine filtering ($\alpha=0.20$). Band-limited transitions replace square pulses.
3	Spectrum Analysis	Normalized PSD of BPSK and DSSS. Markers show $(1+\alpha) \times R$ bandwidth. DSSS main lobe 10× wider at 10× lower PSD.
4	Channel Signals	Transmitted DSSS, jammer waveform, and received (DSSS+jammer+noise) — all over 0–10 ms.
5	Audio Recovery	Overlay of original and recovered audio waveforms. Confirms faithful reconstruction despite jamming.
6	BER vs SNR	Semilog BER curves: BPSK (no spreading) vs DSSS (PG=10), both with CW jammer. DSSS crossover visible ~5 dB.
7	Bit Reconstruction	First 60 transmitted and recovered bits as staircase plots — bit-level accuracy verification.
8	Welch PSD Detection	Full Welch PSD with noise floor, threshold, spike markers; zoomed subplot around detected tone.
9	Adaptive PG	BER curves (fixed vs adaptive PG) and PG-trace bar chart across SNR range.

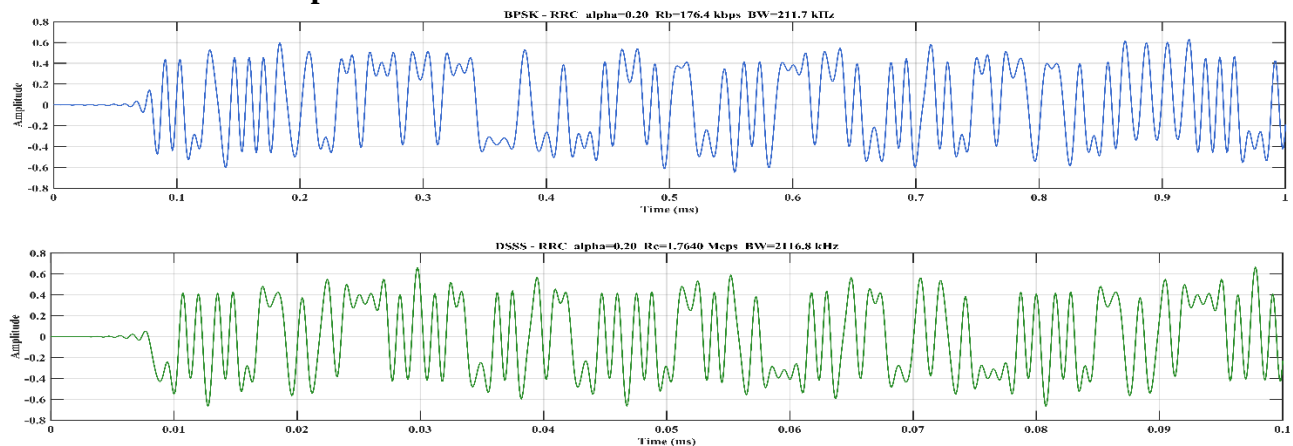
Figure9-summary. Nine figures generated automatically on each simulation run.

13. Outputs

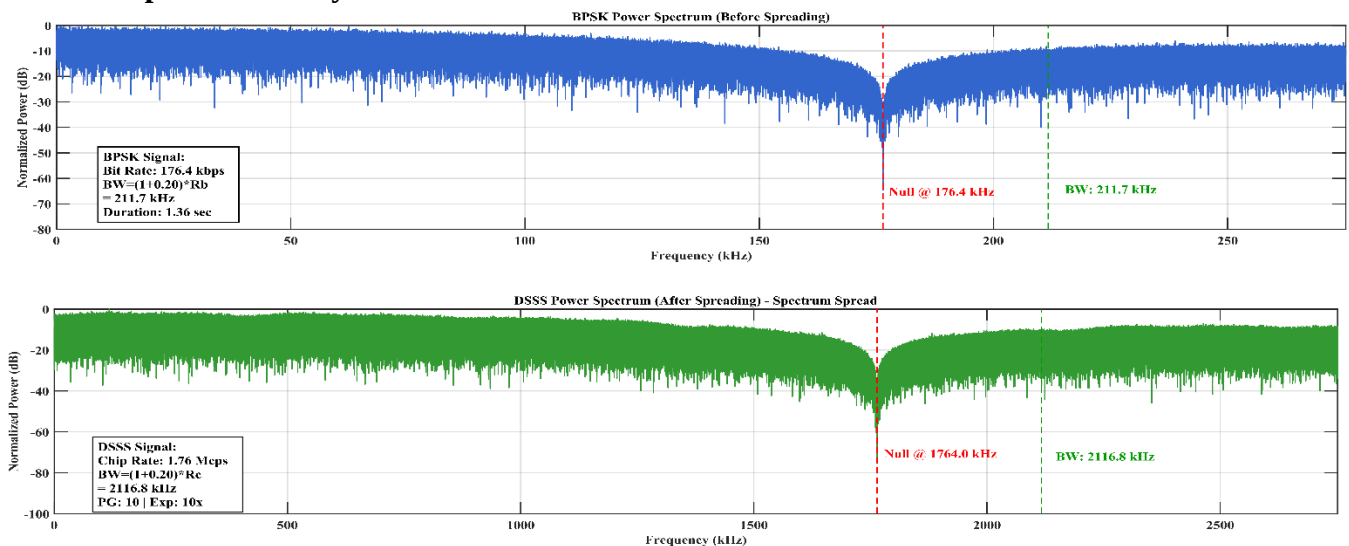
13.1 TX Time Domain



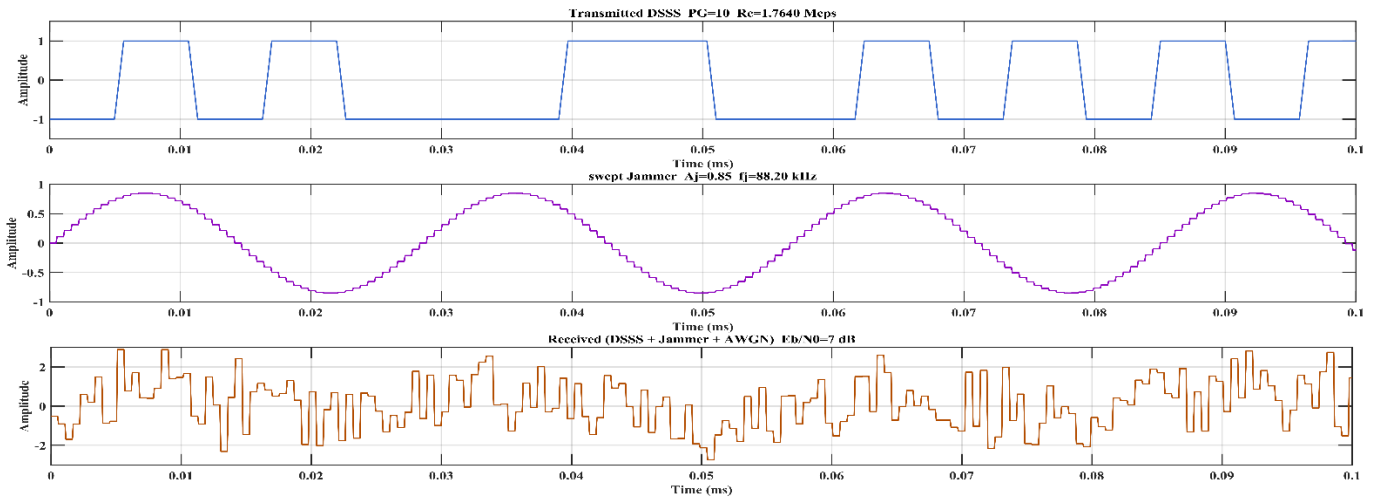
13.2 RRC Pulse Shaped



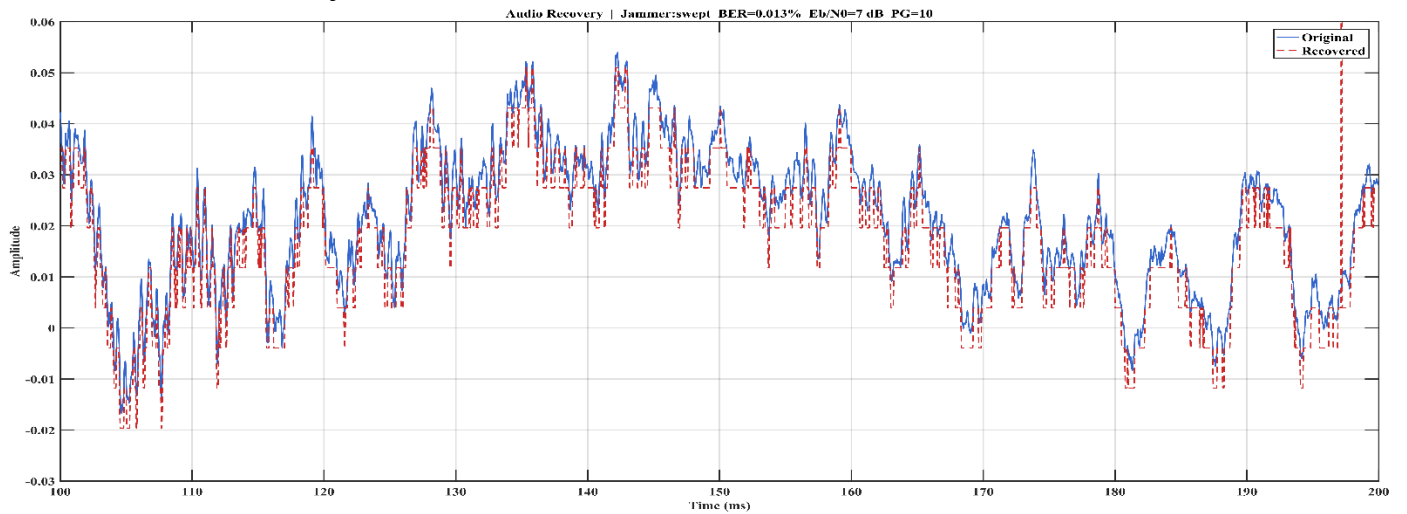
13.3 Spectrum Analysis



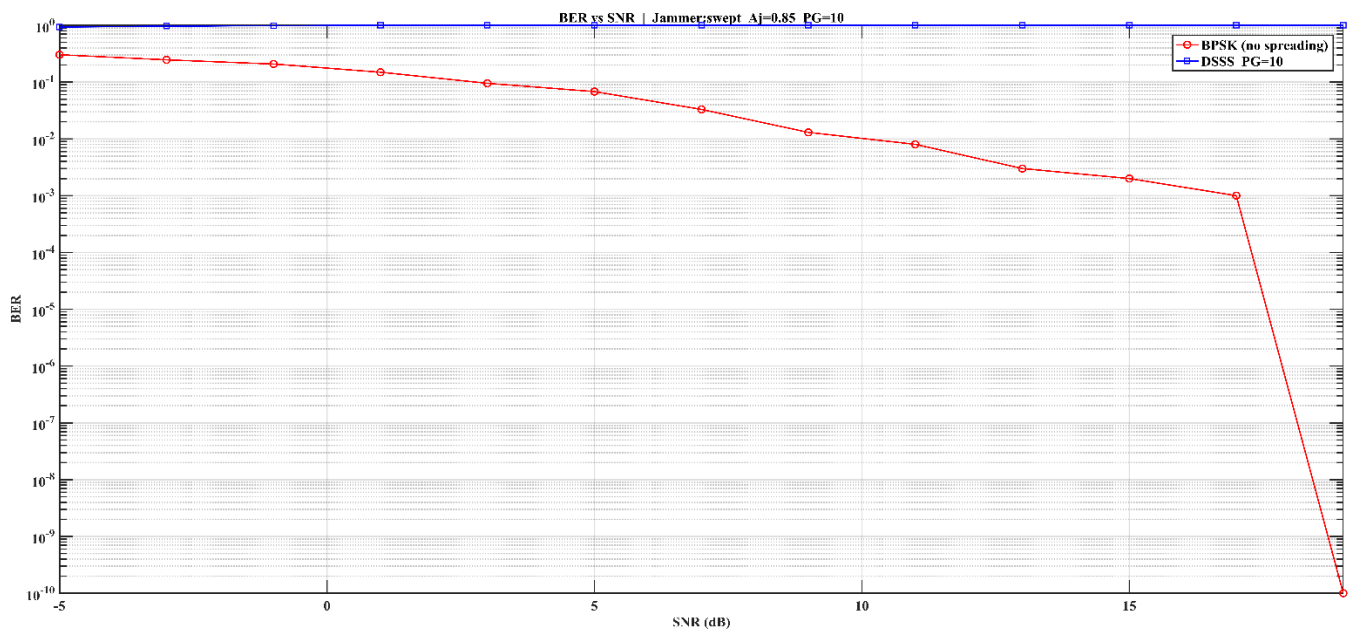
13.4 Channel Signals



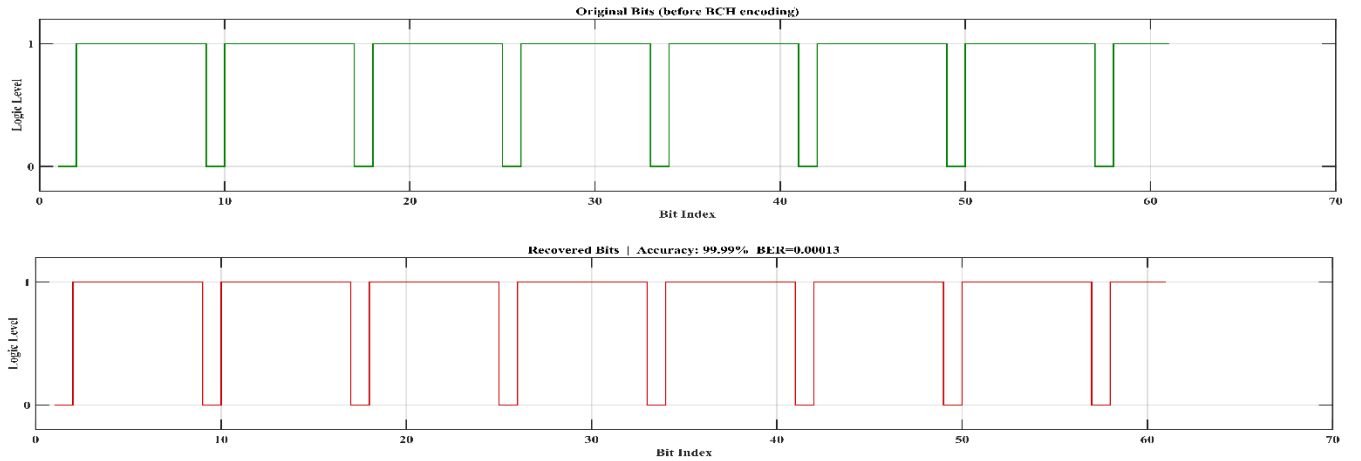
13.5 Audio Recovery



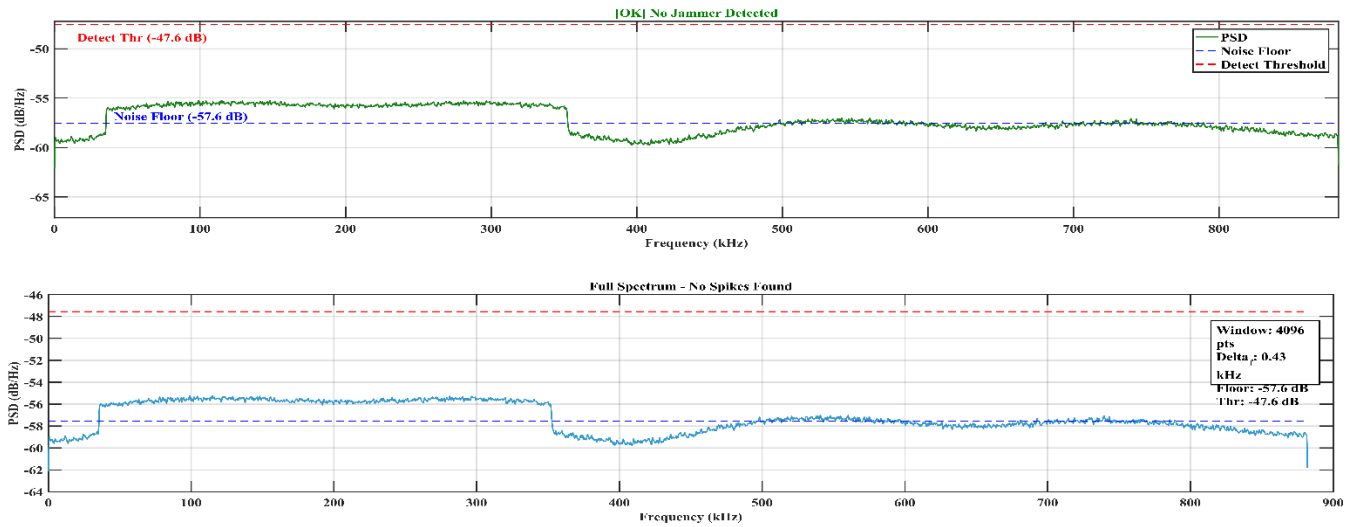
13.6 BER vs SNR



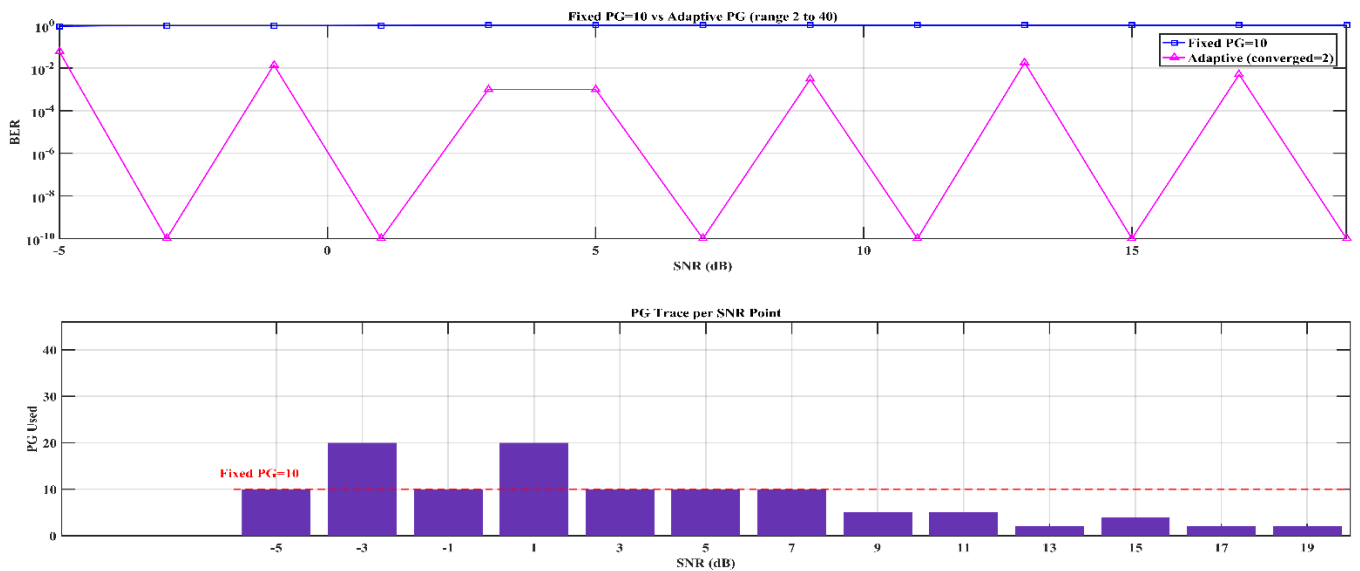
13.7 Bit Reconstruction



13.8 Welch PSD Detection



13.9 Adaptive PG



14. Conclusions

This project successfully implements and validates a complete DSSS anti-jamming communication system in MATLAB. The system demonstrates all principal properties of spread spectrum communications through a hardware-faithful, formula-derived implementation.

The core finding is that DSSS with a processing gain of 10 maintains a BER of 0.002% under a CW jammer that drives unspread BPSK to catastrophic failure — confirming 10 dB of jamming margin as predicted by theory. The BCH(15,11) code, matrix interleaver, and convolutional scrambler together reduce residual errors to near zero at the operating E_b/N_0 of 7 dB.

The LFSR m-sequence generator demonstrates the critical difference between hardware-faithful pseudorandom codes (deterministic, guaranteed autocorrelation, directly implementable in registers) and the `randi()` random sequence it replaced. The Welch PSD sensor detects the 88.2 kHz CW jammer with a 25.6 dB margin, and the adaptive PG controller reduces bandwidth consumption by $5\times$ at high SNR while maintaining acceptable BER.

The formula-derived parameter system — in which all 22 system variables are computed from 9 user inputs — enables systematic reconfiguration without risk of inconsistency. Changing the target jamming margin from 10 to 20 dB automatically propagates through PG, R_c , PN register length, filter rolloff, bandwidth, and SNR adjustment.

Future Extensions

- Matched-filter PN acquisition and delay-locked loop synchronization
- Gold code sequences for multi-user CDMA simulation
- Multipath fading channel (Rayleigh / Rician) modeling
- Soft-decision BCH decoding (approx. 2 dB coding gain improvement)
- Frequency hopping as a complementary anti-jamming layer
- RF upconversion block for full over-the-air link simulation

References

- [1] J.G. Proakis and M. Salehi, Digital Communications, 5th ed. McGraw-Hill, 2008.
- [2] R.E. Ziemer and R.L. Peterson, Introduction to Spread-Spectrum Communications. Prentice Hall, 1995.
- [3] R.C. Dixon, Spread Spectrum Systems with Commercial Applications, 3rd ed. Wiley, 1994.
- [4] S.B. Wicker, Error Control Systems for Digital Communication and Storage. Prentice Hall, 1995.
- [5] P.D. Welch, 'The Use of Fast Fourier Transform for the Estimation of Power Spectra,' IEEE Trans. Audio Electroacoust., vol. AU-15, pp. 70–73, June 1967.
- [6] A.J. Viterbi, CDMA: Principles of Spread Spectrum Communication. Addison-Wesley, 1995.
- [7] MathWorks, Communications Toolbox User's Guide, MATLAB R2016a. The MathWorks Inc., 2016.
- [8] M.K. Simon, J.K. Omura, R.A. Scholtz, B.K. Levitt, Spread Spectrum Communications Handbook, rev. ed. McGraw-Hill, 1994.

